

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Sistemas Operativos I

Trabajo Práctico Final

Integrantes:

Lautaro Capezio
Luciano Duarte

Resumen

Sistema P2P descentralizado en Erlang para compartir archivos en LAN. Utiliza **procesos independientes con paso de mensajes**, evitando mutexes y estado compartido. Análisis de concurrencia, robustez y comparación con C.

Índice

1. Diseño de Concurrencia	2
1.1. Modelo: Procesos Independientes con Paso de Mensajes	2
1.2. Exclusión Mutua Implícita	2
1.3. Prevención de Deadlocks	2
2. Robustez	3
2.1. Estrategias de Manejo de Errores	3
2.2. Supervisor Pattern	3
2.3. Timeouts Configurables	3
3. Comparación: Erlang vs. C	4
3.1. Seguridad de Concurrencia	4
3.2. Deadlocks en C	4
3.3. Manejo de Errores	4
3.4. Escalabilidad	4
3.4.1. Erlang	4
3.4.2. C	5
4. Justificación de Decisiones	5
4.1. ¿Por qué Erlang?	5
4.2. ¿Dónde sería mejor C?	5
5. Conclusiones	5
5.1. Logros	5

1. Diseño de Concurrency

1.1. Modelo: Procesos Independientes con Paso de Mensajes

El sistema implementa procesos Erlang ligeros independientes:

- **node_registry**: Registro de nodos activos
- **download_manager**: Gestión de descargas multi-fuente
- **tcp_server**: Servidor TCP para conexiones entrantes
- **discovery**: Consenso distribuido de IDs únicos
- **hello_broadcast**: Beacons UDP periódicos

Comunicación mediante paso de mensajes asincrónico:

```
node_registry ! {add_node, NodeId, Ip, Port},

receive
  {add_node, NodeId, Ip, Port} ->
    NewNodes = maps:put(NodeId, NodeInfo, Nodes),
    loop(NewNodes)
end.
```

Ventajas: Sin mutexes, sin race conditions, atomicidad implícita.

1.2. Exclusión Mutua Implícita

Erlang **no requiere locks**:

- Atomicidad: Procesamiento de cada mensaje es atómico
- Encapsulamiento: Cada proceso mantiene estado privado
- Orden FIFO: Mensajes se procesan secuencialmente

En `node_registry`, actualizar estado es trivial:

```
NewNodes = maps:put(NodeId, NodeInfo, Nodes),
loop(NewNodes) % Transición atómica
```

1.3. Prevención de Deadlocks

Erlang evita deadlocks por arquitectura:

- Sin locks $\Rightarrow$ sin ciclos de espera
- Cada proceso es dueño de su estado
- Dependencias entre procesos explícitas

Prevención de bloqueos de red: Timeouts en todos los puntos:

```
receive
{response, Data} -> process(Data)
after 5000 ->
{error, timeout}
end.
```

Cleanup automático: Nodos inactivos se eliminan cada 30s (timeout 45s).

2. Robustez

2.1. Estrategias de Manejo de Errores

1. Manejo en Capas

```
handle_client_safe(Socket) ->
try handle_client(Socket)
catch error:Reason -> gen_tcp:close(Socket)
end.
```

2. Detección de Desconexiones El servidor TCP detecta:

- {error, closed}: Cliente cerró conexión
- {error, timeout}: Sin datos en 30s
- {error, Reason}: Otros errores

3. Recuperación por Chunks Archivos >4MB: Descarga por chunks de 4MB

- Si falla un chunk, solo se reintenta ese
- Permite múltiples fuentes en paralelo
- Verificación SHA256 después

2.2. Supervisor Pattern

```
SupFlags = #{strategy => one_for_one, intensity => 5, period => 60}
```

Si un proceso falla >5 veces en 60s, se rinde pero otros continúan.

2.3. Timeouts Configurables

```
-define(ID_REQUEST_TIMEOUT, 10000). % 10s
-define(NODE_TIMEOUT_SECONDS, 45). % 45s sin HELLO
-define(CLEANUP_INTERVAL, 30000). % 30s entre limpiezas
-define(SEARCH_TIMEOUT, 10000). % 10s búsqueda
-define(DOWNLOAD_TIMEOUT, 300000). % 300s descarga
-define(CHUNK_TIMEOUT, 30000). % 30s por chunk
```

3. Comparación: Erlang vs. C

3.1. Seguridad de Concurrency

Aspecto	Erlang	C
Deadlocks	Imposibles	Riesgo alto
Race Conditions	Eliminadas	Requiere expertise
Locks Explícitos	No	Sí (pthread_mutex)
Overhead Creación	Microsegundos	Milisegundos
Máximo Práctico	Millones procesos	Miles threads

3.2. Deadlocks en C

Con múltiples locks (registry + download_manager), riesgo de ciclo:

```
// Thread A: obtiene lock_registry, luego lock_download
// Thread B: obtiene lock_download, luego lock_registry
// RESULTADO: Deadlock circular
```

En Erlang, imposible por arquitectura.

3.3. Manejo de Errores

Erlang: “Let it Crash”

- Errores no se manejan defensivamente
- Supervisores detectan fallos y reinician
- Código limpio sin if/else por cada error

C: Defensivo Manual

- Toda función puede fallar, retorna código error
- Requiere verificación exhaustiva
- Fácil olvidar un chequeo $\Rightarrow$ bug

Ejemplo C:

```
if (recv_request(socket) < 0) return -1;
if (parse_request() < 0) return -1;
if (send_response(socket) < 0) return -1;
// Demasiado verboso y error-prone
```

3.4. Escalabilidad

3.4.1. Erlang

Con 10,000 descargas simultáneas:

- 3MB de memoria (procesos ligeros)
- Sin problema

3.4.2. C

Con 10,000 threads:

- 10-20 GB de memoria (1-2MB por thread)
- Context switching costoso

Comparativa:

	Erlang	C pthread
Overhead Memoria	300 bytes	1-2 MB
Tiempo Creación	Microsegundos	Milisegundos
Context Switch	Muy barato	Caro

4. Justificación de Decisiones

4.1. ¿Por qué Erlang?

1. **Distribución Descentralizada:** Diseñado para sistemas distribuidos; paso de mensajes idéntico local/remoto
2. **Descargas Multi-Fuente:** Cada chunk es su propio proceso; en C requeriría thread pool manual
3. **Resiliencia:** Supervisores reinician automáticamente; en C sería watchdog threads manuales
4. **Sin Deadlocks:** Arquitectura previene ciclos de locks

4.2. ¿Dónde sería mejor C?

1. **Performance CPU:** C más rápido; sistema es I/O-bound
2. **Memoria Embebida:** Control fino; no aplica aquí
3. **Syscalls Directas:** Acceso directo al SO; Erlang abstraer

5. Conclusiones

5.1. Logros

- **Concurrencia Segura:** Sin deadlocks ni race conditions
- **Robustez:** Manejo de fallos con recuperación automática
- **Escalabilidad:** Miles de conexiones simultáneas
- **Claridad:** Código mantenible y sin protecciones deficientes